GRU#

https://pytorch.org/docs/stable/generated/torch.nn.GRU.html#torch.nn.GRU

Description:

Apply a multi-layer gated recurrent unit (GRU) RNN to an input sequence. For each element in the input sequence, each layer computes the following function: where hth_tht is the hidden state at time t, xtx_txt is the input at time t, h(t−1)h_{(t-1)}h(t−1) is the hidden state of the layer at time t-1 or the initial hidden state at time 0, and rtr_trt, ztz_tzt, ntn_tnt are the reset, update, and new gates, respectively. σ\sigmaσ is the sigmoid function, and ⊙\odot⊙ is the Hadamard product. In a multilayer GRU, the input xt(l)x^{(l)}_txt(l) of the lll -th layer (l≥2l \ge 2l≥2) is the hidden state ht(l−1)h^{(l-1)}_tht(l−1) of the previous layer multiplied by dropout δt(l−1)\delta^{(l-1)}_tδt(l−1) where each δt(l−1)\delta^{(l-1)}_tδt(l−1) is a Bernoulli random variable which is 000 with probability dropout. input_size – The number of expected features in the input x hidden_size – The number of features in the hidden state h

Function Signature

```
class torch.nn.GRU(input_size, hidden_size, num_layers=1,
bias=True, batch_first=False, dropout=0.0,
bidirectional=False, device=None, dtype=None)[source]#

Parameters

Inputs: input, h_0

Outputs: output, h_n

Variables
```

Code Examples

```
>>> rnn = nn.GRU(10, 20, 2)
>>> input = torch.randn(5, 3, 10)
>>> h0 = torch.randn(2, 3, 20)
>>> output, hn = rnn(input, h0)
```

Notes & Warnings

NOTE

Note All the weights and biases are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1

NOTE Note For bidirectional GRUs, forward and backward are directions 0 and 1 respectively. Example of splitting the output layers when batch_first=False: output.view(seq_len, batch, num_directions, hidden_size).

NOTE Note batch_first argument is ignored for unbatched inputs.

NOTE Note The calculation of new gate ntn_tnt subtly differs from the original paper and other frameworks. In the original implementation, the Hadamard product (⊙)(\odot)(⊙) between rtr_trt and the previous hidden state h(t−1)h_{(t-1)}h(t−1) is done before the multiplication with the weight matrix W and addition of bias: nt=tanh(Winxt+bin+Whn(rt⊙h(t−1))+bhn)\begin{aligned} n_t = \tanh(W_{in} x_t + b_{in} + W_{hn} (r_t \odot h_{(t-1)}) + b_{hn}) \end{aligned} nt =tanh(Winxt+bin+Whn(rt⊙h(t−1))+bhn)This is in contrast to PyTorch implementation, which is done after Whnh(t−1)W_{hn} h_{(t-1)}Whnh(t−1) nt=tanh(Winxt+bin+rt⊙(Whnh(t−1)+bhn))\begin{aligned} n_t = \tanh(W_{in} x_t + b_{in} + r_t \odot (W_{hn} h_{(t-1)}+ b_{hn})) \end{aligned} nt =tanh(Winxt+bin+rt⊙(Whnh(t

NOTE Note If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype torch.float16 4) V100 GPU is used, 5) input data is not in PackedSequence format persistent algorithm can be selected to improve performance.

Detailed Documentation

Documentation

Apply a multi-layer gated recurrent unit (GRU) RNN to an input sequence. For each element in the input sequence, each layer computes the following function:

where hth_tht is the hidden state at time t, xtx_txt is the input at time t, h(t−1)h_{(t-1)}h(t−1) is the hidden state of the layer at time t-1 or the initial hidden state at time 0, and rtr_trt, ztz_tzt, ntn_tnt are the reset, update, and new gates, respectively. σ\sigmaσ is the sigmoid function, and ⊙\odot⊙ is the Hadamard product.

In a multilayer GRU, the input xt(l)x^{(l)}_txt(l) of the lll -th layer (l≥2l \ge 2l≥2) is the hidden state ht(l−1)h^{(l-1)}_tht(l−1) of the previous layer multiplied by dropout δt(l−1)\delta^{(l-1)}_tδt(l−1) where each δt(l−1)\delta^{(l-1)}_tδt(l−1) is a Bernoulli random variable which is 000 with probability dropout.

input_size – The number of expected features in the input x

hidden_size – The number of features in the hidden state h

num_layers – Number of recurrent layers. E.g., setting num_layers=2 would mean stacking two GRUs together to form a stacked GRU, with the second GRU taking in outputs of the first GRU and computing the final results. Default: 1

bias – If False, then the layer does not use bias weights b_ih and b_hh. Default: True

batch_first – If True, then the input and output tensors are provided as (batch, seq, feature) instead of (seq, batch, feature). Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: False

dropout – If non-zero, introduces a Dropout layer on the outputs of each GRU layer except the last layer, with dropout probability equal to dropout. Default: 0

bidirectional – If True, becomes a bidirectional GRU. Default: False

input: tensor of shape (L,Hin)(L, H_{in})(L,Hin) for unbatched input, (L,N,Hin)(L, N, H_{in})(L,N,Hin) when batch_first=False or (N,L,Hin)(N, L, H_{in})(N,L,Hin) when batch_first=True containing the features of the input sequence. The input can also be a packed variable length sequence. See torch.nn.utils.rnn.pack_padded_sequence() or torch.nn.utils.rnn.pack_sequence() for details.

h_0: tensor of shape (D∗num_layers,Hout)(D * \text{num_layers}, H_{out})(D∗num_layers,Hout) or (D∗num_layers,N,Hout)(D * \text{num_layers}, N, H_{out})(D∗num_layers,N,Hout) containing the initial hidden state for the input sequence. Defaults to zeros if not provided.

output: tensor of shape (L,D∗Hout)(L, D * H_{out})(L,D∗Hout) for unbatched input, (L,N,D∗Hout)(L, N, D * H_{out})(L,N,D∗Hout) when batch_first=False or (N,L,D∗Hout)(N, L, D * H_{out})(N,L,D∗Hout) when batch_first=True containing the output features (h_t) from the last layer of the GRU, for each t. If a torch.nn.utils.rnn.PackedSequence has been given as the input, the output will also be a packed sequence.

h_n: tensor of shape (D∗num_layers,Hout)(D * \text{num_layers}, H_{out})(D∗num_layers,Hout) or (D∗num_layers,N,Hout)(D * \text{num_layers}, N, H_{out})(D∗num_layers,N,Hout) containing the final hidden state for the input sequence.

weight_ih_l[k] – the learnable input-hidden weights of the kth\text{k}^{th}kth layer (W_ir|W_iz|W_in), of shape (3*hidden_size, input_size) for k = 0. Otherwise, the shape is (3*hidden_size, num_directions * hidden_size)

weight_hh_l[k] – the learnable hidden-hidden weights of the kth\text{k}^{th}kth layer (W_hr|W_hz|W_hn), of shape (3*hidden_size, hidden_size)

bias_ih_l[k] – the learnable input-hidden bias of the kth\text{k}^{th}kth layer (b_ir|b_iz|b_in), of shape (3*hidden_size)

bias_hh_l[k] – the learnable hidden-hidden bias of the kth\text{k}^{th}kth layer (b_hr|b_hz|b_hn), of shape (3*hidden_size)

All the weights and biases are initialized from U(–k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(–k ,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1

For bidirectional GRUs, forward and backward are directions 0 and 1 respectively. Example of splitting the output layers when batch_first=False: output.view(seq_len, batch, num_directions, hidden_size).

batch_first argument is ignored for unbatched inputs.

The calculation of new gate ntn_tnt subtly differs from the original paper and other frameworks. In the original implementation, the Hadamard product (⊙)(\odot)(⊙) between rtr_trt and the previous hidden state h(t–1)h_{(t-1)}h(t–1) is done before the multiplication with the weight matrix W and addition of bias:

This is in contrast to PyTorch implementation, which is done after Whnh(t–1)W_{hn} h_{(t-1)}Whnh(t–1)

This implementation differs on purpose for efficiency.

If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype torch.float16 4) V100 GPU is used, 5) input data is not in PackedSequence format persistent algorithm can be selected to improve performance.